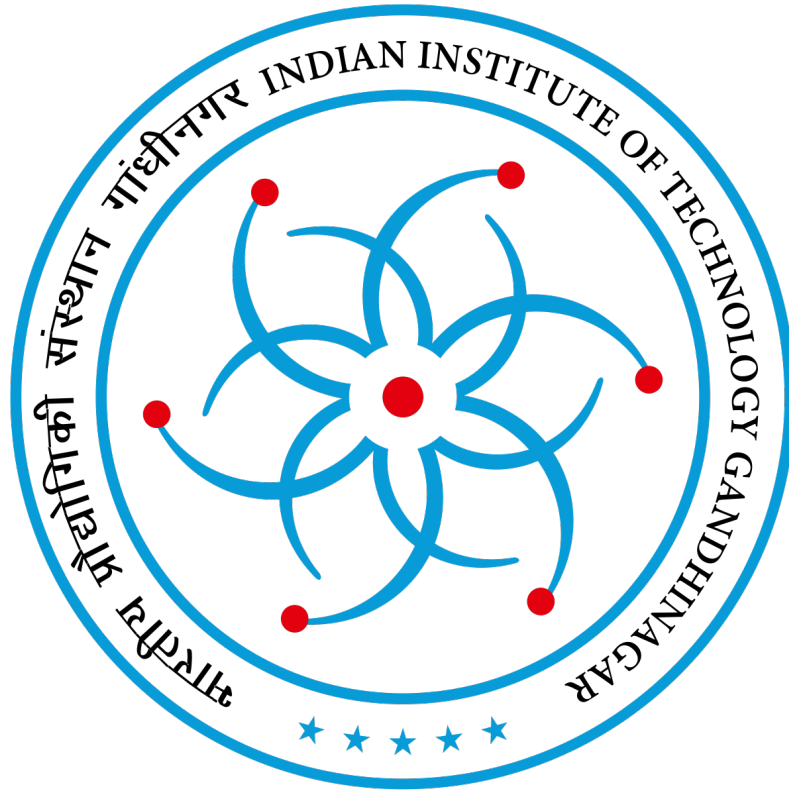


# Indian Institute of Technology Gandhinagar



## Computer Solution of 2-D Steady Heat Diffusion Equation

**ME:605 Computational Fluid Dynamics**

**Instructor:** *Prof. Dilip Srinivas Sundaram*

**Teaching Assistants:** *Pardha Sai Mutyala, Soham Pathak*

**Name:** Atharva Suryarao

**Roll No:** 23110051

**Date of Submission:** 02/10/2025

**Code Link:** [Click here](#)

# Contents

|          |                                                                                                          |          |
|----------|----------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Problem Statement</b>                                                                                 | <b>3</b> |
| 1.1      | Boundary Conditions . . . . .                                                                            | 3        |
| <b>2</b> | <b>Mesh Details</b>                                                                                      | <b>3</b> |
| 2.1      | Mesh Type . . . . .                                                                                      | 3        |
| 2.2      | Grid Points . . . . .                                                                                    | 4        |
| 2.3      | Domain Size . . . . .                                                                                    | 4        |
| 2.4      | Grid Spacing . . . . .                                                                                   | 4        |
| <b>3</b> | <b>Methodology for Discretization</b>                                                                    | <b>4</b> |
| 3.1      | Discretization of Governing Equation . . . . .                                                           | 4        |
| 3.2      | Dirichlet Boundary Conditions . . . . .                                                                  | 5        |
| 3.3      | Neumann boundary condition . . . . .                                                                     | 5        |
| <b>4</b> | <b>Solution Methodology</b>                                                                              | <b>6</b> |
| 4.1      | Gauss Elimination Method . . . . .                                                                       | 6        |
| 4.2      | Applying the Gauss Elimination Method . . . . .                                                          | 7        |
| 4.2.1    | Creating the Matrix A and Vector B . . . . .                                                             | 7        |
| 4.2.2    | Temperature Field for $N = 41$ grid points . . . . .                                                     | 7        |
| 4.2.3    | Comparison of CPU Run Time for $N = 11, 21, 41$ . . . . .                                                | 8        |
| 4.2.4    | Computational Efficiency of Gauss Elimination Method . . . . .                                           | 9        |
| 4.3      | Gauss Seidel Method . . . . .                                                                            | 9        |
| 4.3.1    | Applying the Gauss Seidel Method . . . . .                                                               | 10       |
| 4.3.2    | Temperature Field for $N = 41$ grid points . . . . .                                                     | 11       |
| 4.3.3    | Calculating the Residual . . . . .                                                                       | 11       |
| 4.3.4    | Residual Versus Number of iterations for $N = 11, 21, 41$ grid points                                    | 12       |
| 4.3.5    | Dependence of Convergence rate on the total number of grid points                                        | 12       |
| 4.3.6    | Comparison of CPU Run Time for $N = 11, 21, 41$ grid points . . .                                        | 13       |
| 4.3.7    | Comparison of CPU Run Time for Gauss-Seidel versus Gauss<br>Elimination method. . . . .                  | 13       |
| 4.4      | Line by Line Method (Row Sweep) . . . . .                                                                | 14       |
| 4.4.1    | Applying the Row Sweep Method . . . . .                                                                  | 14       |
| 4.4.2    | Temperature Distribution for $N = 41$ Grid Points . . . . .                                              | 15       |
| 4.4.3    | Comparison of CPU Run Time for $N = 11, 21, 41$ grid points . . .                                        | 16       |
| 4.4.4    | Comparison of CPU Run Time for Row Sweep versus Gauss Elim-<br>ination method. . . . .                   | 16       |
| 4.4.5    | Comparison of CPU Run Time for Row Sweep versus Gauss Seidel<br>method. . . . .                          | 17       |
| 4.4.6    | Calculating the Residual . . . . .                                                                       | 17       |
| 4.4.7    | Comparison of Residual versus number of iterations of Gauss-<br>Seidel and Line by Line method . . . . . | 18       |

|          |                                                                                                            |           |
|----------|------------------------------------------------------------------------------------------------------------|-----------|
| 4.5      | Alternating Direction Implicit (ADI) Method . . . . .                                                      | 18        |
| 4.5.1    | Comparison of Residual versus number of iterations for row sweep,<br>column sweep and ADI method . . . . . | 19        |
| <b>5</b> | <b>Concluding Remarks</b>                                                                                  | <b>19</b> |
| <b>6</b> | <b>Acknowledgments</b>                                                                                     | <b>20</b> |
| <b>7</b> | <b>References</b>                                                                                          | <b>20</b> |

# 1 Problem Statement

The aim of this project is to get the numerical solution of an Elliptic PDE which in our case is the Two Dimensional Heat diffusion equation and get the solution using different approaches and Algorithms and comment on the efficiency of one method over the other.

The governing differential equation for the problem statement is:-

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

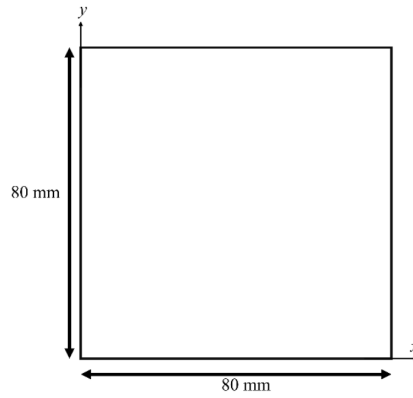


Figure 1: Square computational domain of size 80 mm  $\times$  80 mm.

## 1.1 Boundary Conditions

The boundary conditions are as follows:-

- $T(x, y = 0) = 323 \text{ K}$
- $T(x, y = 80 \text{ mm}) = 423 \text{ K}$
- $T(x = 80 \text{ mm}, y) = 473 \text{ K}$
- $h(T_\infty - T(x = 0, y)) = -\lambda \frac{\partial T}{\partial x} \Big|_{x=0}$
- $h = 250 \frac{\text{W}}{\text{m}^2\text{-K}}, \quad \lambda = 5 \frac{\text{W}}{\text{m-K}} \quad \text{and} \quad T_\infty = 573 \text{ K}$

## 2 Mesh Details

### 2.1 Mesh Type

The type of grid used for the two-dimensional heat diffusion equation is a **Structured Grid**.

## 2.2 Grid Points

The number of Grid points in this computation ranges from 11, 21 and 41.

## 2.3 Domain Size

The domain size is the dimensions of the domain in which we are interested in calculating the temperatures. For this project, the domain size is 80 mm x 80 mm.

## 2.4 Grid Spacing

The grid spacing is the distance between two adjacent grid points. In this case, we have a structured and symmetric grid, meaning the grid spacing in the x and y directions is equal in magnitude.

The formula for calculating the grid spacing is as follows:-

- Firstly, discretize the mesh into smaller grids of size  $\Delta x$  and  $\Delta y$ .

$$\Delta x = \frac{\text{Domain Size}}{N - 1}, \Delta y = \frac{\text{Domain Size}}{N - 1}$$

(here N is the number of grid points and the size of the domain is the length of the domain).

# 3 Methodology for Discretization

## 3.1 Discretization of Governing Equation

The governing Partial Differential equation for the 2-D steady-state heat diffusion equation is shown below.

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Using the **Finite Difference Method**, we can convert the partial differential equations into algebraic equations, and to increase the order of accuracy, we explicitly use the **Central Difference method**, which is a **second-order** accurate approximation.

$$\frac{d^2 \phi}{dx^2} = \frac{\phi(x_{i+1}) - 2\phi(x_i) + \phi(x_{i-1}))}{\Delta x^2} + O(\Delta x^2)$$
$$\frac{d^2 \phi}{dy^2} = \frac{\phi(y_{i+1}) - 2\phi(y_i) + \phi(y_{i-1}))}{\Delta y^2} + O(\Delta y^2)$$

We add these equations together ignoring the error term, and since  $\Delta x = \Delta y$ , we get the following expression .

$$\frac{\phi(x_{i+1}) - 2\phi(x_i) + \phi(x_{i-1}))}{\Delta x^2} + \frac{\phi(y_{i+1}) - 2\phi(y_i) + \phi(y_{i-1}))}{\Delta y^2} = 0$$

$$\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j} + \phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1} = 0$$

$$\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} = 4\phi_{i,j}$$

$$\phi_{i,j} = \frac{1}{4}(\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1})$$

In our case as we are calculating the temperatures the final discretised equation will be.

$$T_{i,j} = \frac{1}{4} (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1})$$

### 3.2 Dirichlet Boundary Conditions

The Boundary Conditions specified at the domain's top, bottom and right boundary are Dirichlet Boundary Conditions, meaning the temperature determined at these boundaries does not change.

### 3.3 Neumann boundary condition

The Boundary Conditions specified at the domain's left boundary are Neumann Boundary Conditions, meaning the temperature at the left boundary will change accordingly.

The governing differential equation at the left boundary is as follows:-

$$h(T_\infty - T(x=0, y)) = -\lambda \left. \frac{\partial T}{\partial x} \right|_{x=0}$$

We use a first-order approximation to convert the following differential equations into an algebraic equation.

As we are at the left boundary, we have no point to the left, but we have a point to the right of the boundary, so instead of using the Central difference scheme, we use the forward difference scheme to discretize the equation.

According to the first-order approximation of  $\frac{\partial T}{\partial x}$  we get:

$$\frac{\partial T}{\partial x} = \frac{T_{i,j+1} - T_{i,j}}{\Delta x}$$

$$h(T_\infty - T(x=0, y)) = -\lambda \frac{T_{i,2} - T_{i,1}}{\Delta x}$$

$$h(T_\infty - T_{i,1}) = -\lambda \frac{T_{i,2} - T_{i,1}}{\Delta x}$$

After simplifying the expression we get the following expression.

$$T_{i,1} = \frac{h\Delta x T_{\infty} + \lambda T_{i,2}}{\lambda + h\Delta x}$$

The variables represent the following physical quantity

- $h$  is the convection heat transfer coefficient.
- $\lambda$  is the thermal conductivity.
- $T_{\infty}$  is the ambient temperature.
- $\Delta x$  is the grid spacing.
- $T_{i,1}$  is the unknown temperature of the left boundary.
- $T_{i,2}$  is the temperature of the grid point to the right of  $T_{i,1}$ .

## 4 Solution Methodology

### 4.1 Gauss Elimination Method

The Gaussian Elimination Method is a widely used technique for solving systems of linear equations, where multiple equations with unknown variables are solved simultaneously to find the values of the unknowns.

A general 3 by 3 matrix looks like this.

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \quad X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, \quad B = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

It is a row reduction algorithm for solving linear systems. It involves a series of operations on the augmented matrix (which includes both coefficients and constants) to simplify it into a row-echelon form or reduced row-echelon form.

The following discretized equation will be of the form  $\mathbf{A} \mathbf{x} = \mathbf{b}$

- $\mathbf{A}$  is the Augmented Matrix.
- $\mathbf{x}$  is the variable that we are interested in.
- $\mathbf{B}$  is the known vector.

After forward elimination the Augmented Matrix and the solution vector for a 3 by 3 Augmented matrix gets converted into the following form.

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{pmatrix}, \quad X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, \quad B = \begin{pmatrix} b'_1 \\ b'_2 \\ b'_3 \end{pmatrix}$$

With Back substitution we can get the values of  $x_1, x_2, x_3$ .

## 4.2 Applying the Gauss Elimination Method

As this method involves solving the equation  $\mathbf{A} \mathbf{x} = \mathbf{b}$  we first need to create the Matrix A and Vector B.

In our case, the variable we are solving for is the **temperature field**. For an  $N \times N$  matrix, the number of points at which the temperature will be calculated is  $N^2$ .

### 4.2.1 Creating the Matrix A and Vector B

Firstly, instead of creating a matrix which has only unknown temperatures such as the temperatures at the interior points and on the left boundary we create a matrix which considers all the temperatures as unknown, also one step that we take is getting the values of global nodes rather than specifying the indices (i,j) this helps in simplifying the logic for creating the matrix A.

$$\text{node} = (i - 1) \cdot N + j, \quad \text{for } i = 1, 2, \dots, N \text{ and } j = 1, 2, \dots, N$$

According to this logic the Vector B & Vector T will be of size  $(N * N) \times 1$  and the Matrix A will be of size  $(N * N) \times (N * N)$ .

- If the point lies on the Top, right and bottom boundary  $A(\text{node}, \text{node}) = 1$  and  $B(\text{node}) = T(i, j)$ .
- If the point lies on the Left Boundary then using the first order Approximation and simplifying we can write  $A(\text{node}, \text{node}) = \lambda + h\Delta x$ ,  $A(\text{node}, \text{node} + 1) = -\lambda$  and  $B(\text{node}) = \lambda + h\Delta x T_\infty$ .
- For points that lie on the interior we will use the discretized governing equation to get the values of coefficients and they will be the following  $A(\text{node}, \text{node}) = -4$ ,  $A(\text{node}, \text{node} = 1) = 1$ ,  $A(\text{node}, \text{node} - 1) = 1$ ,  $A(\text{node} + 1, \text{node}) = 1$ ,  $A(\text{node} - 1, \text{node}) = 1$  and  $B(\text{node}) = 0$

Looping the logic over the Matrix A and Vector B will be created. Once our Matrix A and Vector B are created, we can calculate the values of our expression using the Gauss Elimination function that we have created before.

### 4.2.2 Temperature Field for N = 41 grid points

The temperature distribution using the Gauss Elimination method for 41 grid points in  $x$  and  $y$  directions has been shown in the next page.



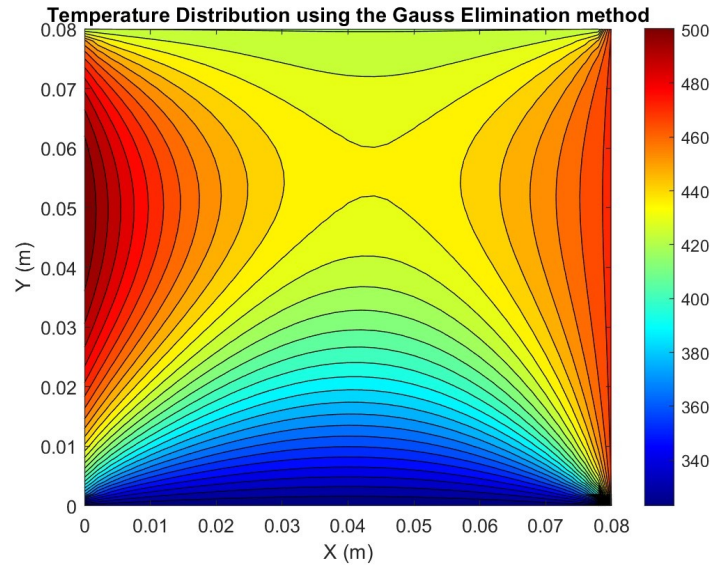


Figure 2: Temperature distribution for  $N = 41$  grid points using Gauss Elimination Method

#### 4.2.3 Comparison of CPU Run Time for $N = 11, 21, 41$

The CPU run time increases exponentially as  $N$  increases and the graph for the same has been shown below.

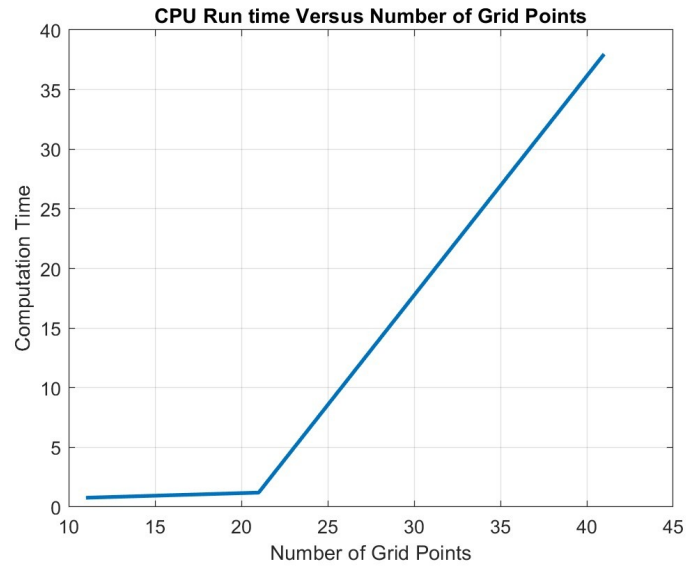


Figure 3: Comparison of CPU Run Time for Different Number of Grid Points

#### 4.2.4 Computational Efficiency of Gauss Elimination Method

- As the number of grid points increases from 11 to 21, we do not observe any substantial increase in the computation time.
- As we increase the number of grid points to 41, we observe a considerable increase in the computational time, and one of the reasons is the increase in the number of rows and columns in the matrix  $A$ , which has a dimension of  $N \times N$  by  $N \times N$  and as  $N$  increases the size of the matrix  $A$  increases by a square of  $N$  and this leads to increased number of calculations which increases the computation time.
- The increase in the computation time overshadows the accurate solution that the Gauss Elimination method gives; hence, this method is not used as real-life problems are more complex, and we need faster methods to get the solution to our problem.

#### 4.3 Gauss Seidel Method

The Gauss-Seidel method, is an iterative method used to solve a system of linear equations. Though it can be applied to any matrix with non-zero elements on the diagonals, convergence is only guaranteed if the matrix is either strictly diagonally dominant or symmetric and positive definite.

A general 3 by 3 matrix looks like this.

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \quad X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, \quad B = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

When  $A$  and  $b$  are known, and  $x$  is unknown, the Gauss-Seidel method can be used to iteratively approximate  $x$ . The vector  $x^{(0)}$  denotes the initial guess for  $x$ , often  $x_i^{(0)} = 0$  for  $i = 1, 2, \dots, n$ . Denote by  $x^{(k)}$  the  $k$ -th approximation (or iteration) of  $x$ , and by  $x^{(k+1)}$  the approximation of  $x$  at the next or  $k + 1$ -th iteration.

In Gauss-Seidel, the computation of  $x^{(k+1)}$  uses the elements of  $x^{(k+1)}$  that have already been computed, and only the elements of  $x^{(k)}$  that have not yet been computed in the  $(k + 1)$ -th iteration. This means that, unlike the Jacobi method, only one storage vector is required since elements can be overwritten as they are computed, which can be advantageous for very large problems.

The convergence properties of the Gauss-Seidel method are dependent on the matrix  $A$ . Namely, the procedure is known to converge if either:

- $A$  is symmetric positive-definite, or
- $A$  is strictly or irreducibly diagonally dominant.

The Gauss-Seidel method may converge even if these conditions are not satisfied.

### 4.3.1 Applying the Gauss Seidel Method

The discretized governing equation is shown below.

$$T_{i,j} = \frac{1}{4} (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1})$$

According to the Gauss Seidel method we do not explicitly need to create the matrix A and Vector B as they will be embedded in the Temperature Matrix.

We first create a zeros matrix of the size  $N$  where  $N$  is the number of grid points and assign the known temperatures to the top, right and bottom boundary. The temperature matrix for 5 grid points will look the matrix depicted below.

$$\mathbf{T} = \begin{bmatrix} 423 & 423 & 423 & 423 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 323 & 323 & 323 & 323 & 473 \end{bmatrix}$$

The initial guess of the temperature at the left boundary and the interior points has been taken to be zero.

The Gauss-Seidel Method calculates the temperatures at individual nodes/points according to the following formula.

$$T_{i,j}^{(n+1)} = \frac{1}{4} (T_{i+1,j}^{(n)} + T_{i-1,j}^{(n+1)} + T_{i,j+1}^{(n)} + T_{i,j-1}^{(n+1)})$$

- $n$  is the previous iteration.
- $n + 1$  is the current iteration.

After the first iteration, the interior nodes that were initially zero will have a different temperature based on the above formula. This process is looped over, and the error is calculated after each step. Once the error is less than the tolerance set by us, the loop stops executing, and we get the temperature distribution.

As we have convective heat loss at the left boundary we will also have to iterate for it using the appropriate formula shown below.

$$T_{i,1}^{n+1} = \frac{h\Delta x T_{\infty} + \lambda T_{i,2}^n}{\lambda + h\Delta x}$$

- $n$  is the previous iteration.
- $n + 1$  is the current iteration.

This process is looped over, and the error is calculated after each step. Once the error is less than the tolerance set by us, the loop stops executing, and we get the temperature distribution.

### 4.3.2 Temperature Field for N = 41 grid points

The temperature distribution using the Gauss-Seidel method for 41 grid points in  $x$  and  $y$  directions has been shown below.

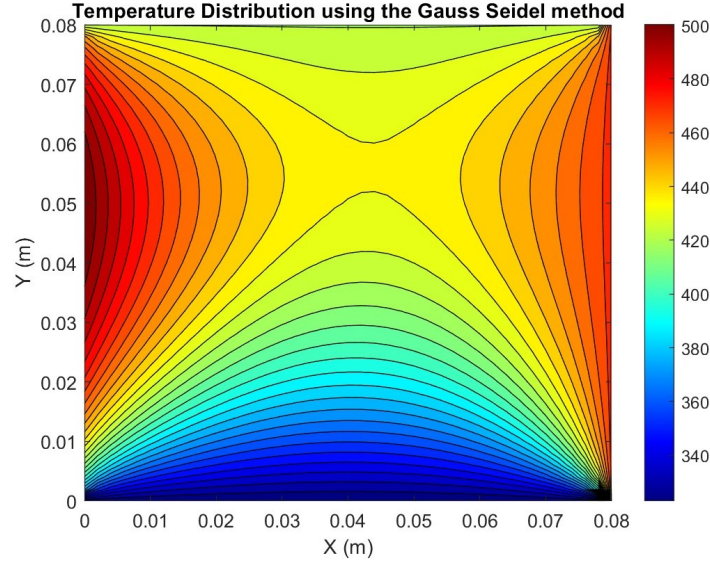


Figure 4: Temperature distribution for N = 41 grid points using Gauss-Seidel Method

### 4.3.3 Calculating the Residual

Calculation of the residual is a crucial step in the iterative process, as it enables us to determine whether our solution is converging or diverging.

There are multiple methods of calculating the residual. One method includes calculating the  $L2$  Norm of the matrix, and the second involves calculating the mean of the difference between the current iteration matrix and the previous iteration matrix.

The method I employed to calculate the residual involves calculating the mean of the difference between the current iteration matrix and the previous iteration matrix, as shown in the formula below.

$$\text{Residual} = \frac{1}{N^2} \sum_{i,j} |T_{i,j}^{(n+1)} - T_{i,j}^{(n)}|$$

- $n$  denotes the previous iteration,  $n + 1$  denotes the current iteration (or time step).
- $N$  denotes the number of grid points.

#### 4.3.4 Residual Versus Number of iterations for $N = 11, 21, 41$ grid points

The residual versus number of iterations for  $N = 11, 21, 41$  grid points has been shown below.

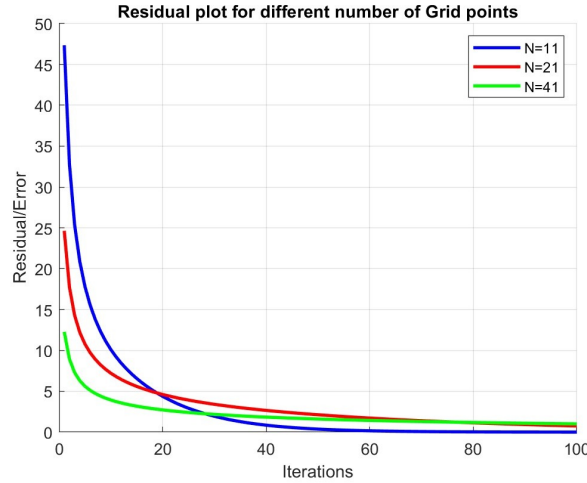


Figure 5: Residual Versus Number of iterations plot

#### 4.3.5 Dependence of Convergence rate on the total number of grid points

- As the total number of grid points increase the number of iterations also increase. The increased number of iterations leads to increased computations by the computer.
- Looking at the graph plotted above, the initial error decreases with an increase in the total number of grid points, but as the iterations increase, the error falls sharply.
- The convergence rate also depends on the tolerance set by the user; in this case, the tolerance is set to  $10^{-6}$ , hence for  $N = 41$  more iterations are required to reach convergence, but if the tolerance is set to  $10^{-3}$ , the solution will converge even faster.
- The convergence rate for  $N = 11$  is greater  $N = 41$  and can be seen through the slope of residual versus iterations graph.
- One important thing to notice here is that after a certain number of iterations for  $N = 41$ , the change in error is almost negligible, hence in such a case, even a tolerance of  $10^{-3}$  would essentially give the same solution and save us a lot of computation time.

#### 4.3.6 Comparison of CPU Run Time for $N = 11, 21, 41$ grid points

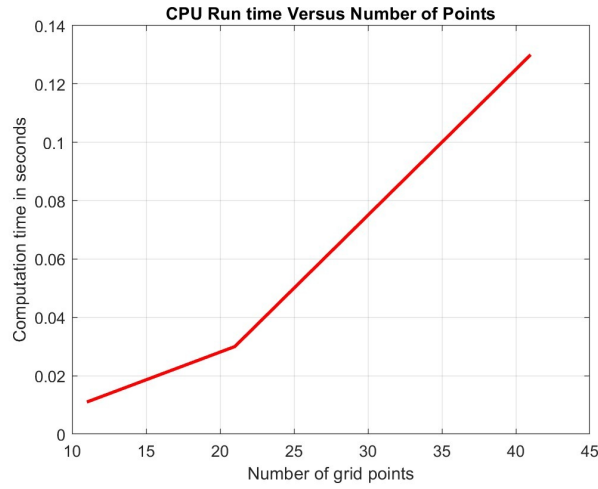


Figure 6: Variation of CPU Run Time with increase in  $N$

Analysing the graph, we can clearly see that the CPU Run time has no significant changes even if we increase the number of points from  $N = 11$  to  $N = 41$ , and the reason behind it is that the computer has to do fewer and simpler calculations when using an iterative approach. Hence, we use iterative methods over direct ones in real-life problems where time is crucial and problems are more complicated.

#### 4.3.7 Comparison of CPU Run Time for Gauss-Seidel versus Gauss Elimination method.

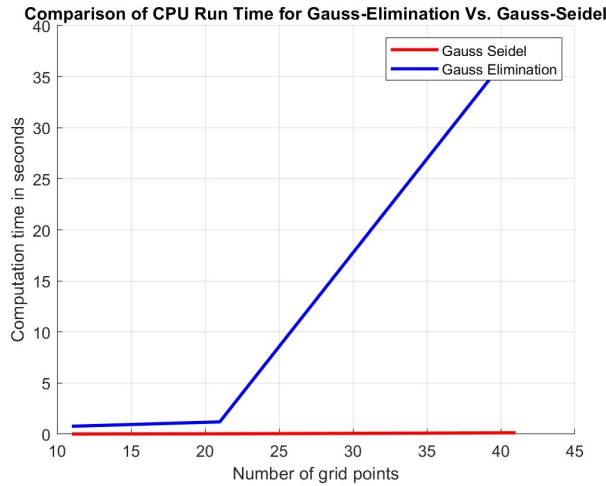


Figure 7: Comparison of CPU Run Time for different methods

Looking at the graph above, it is very clear that Gauss Elimination is an inefficient method in computing the solution, as for even smaller  $N$ , the time it takes is much more than Gauss-Seidel. Hence, the Gauss-Seidel method is a better method for getting the solution.

#### 4.4 Line by Line Method (Row Sweep)

The Row Sweep method is an efficient way of solving the 2D heat diffusion equation. In this method instead of calculating the temperature at a single point we calculate the temperature of all points that lie in a row simultaneously using the TDMA (Thomas Diagonal Matrix Algorithm) solver.

The row sweep method is useful for problems with regular geometries and boundary conditions and in cases of a rectangular domain or a square domain we can use this method to solve our problem.

##### 4.4.1 Applying the Row Sweep Method

The discretized governing equation is shown below.

$$T_{i,j} = \frac{1}{4} (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1})$$

In the case of Row sweep method, we rearrange the terms in the equation to get the following expression.

$$4T_{i,j}^{n+1} - (T_{i,j+1}^{n+1} + T_{i,j-1}^{n+1}) = T_{i+1,j}^n + T_{i-1,j}^n$$

- $n$  denotes the previous iteration (or time step).
- $n + 1$  denotes the current iteration (or time step).

For the row sweep method, as we will be solving the linear system of equations using the TDMA solver, we need to first assign random values to the interior nodes and the left boundary nodes; the remaining temperatures will be determined by the boundary temperatures provided to us. The temperature matrix for 5 grid points will look like the matrix depicted below.

$$\mathbf{T} = \begin{bmatrix} 423 & 423 & 423 & 423 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 0 & 0 & 0 & 0 & 473 \\ 323 & 323 & 323 & 323 & 473 \end{bmatrix}$$

To get the temperature of the interior nodes, let us start with the second row. To get the values of temperatures, we will have to assemble the Matrix  $A$  and Vector  $B$ , where  $A$  will be a tridiagonal matrix.

Lets look at the logic for creating the Matrix  $A$  and Vector  $B$ .

- The size of the Matrix A will be  $N \times N$  and the vector B will be  $N \times 1$ .
- If we are at the interior nodes, which have an index  $j$ , we can write from the row sweep equation that  $A(j, j) = 4$ ,  $A(j, j - 1) = -1$ , and  $A(j, j + 1) = -1$  and  $B(j) = T_{i+1,j}^n + T_{i-1,j}^n$ .
- If we are the right boundary, we know the temperature at that point and we can write  $A(j, j) = 1$  and  $B(j) = T_{right}$ .
- If we are at the left boundary, we have a convective condition and from the discretized equation mentioned previously for the left boundary we can write  $A(j, j) = \lambda + h\Delta x$ ,  $A(j, j + 1) = -\lambda$  and  $B(j) = h\Delta x T_\infty$ .

The matrix for a row will look like the matrix shown below.

$$\begin{bmatrix} \lambda + h\Delta x & -\lambda & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} T(i, 1) \\ T(i, 2) \\ T(i, 3) \\ T(i, 4) \\ T(i, 5) \end{bmatrix} = \begin{bmatrix} h\Delta x T_\infty \\ T^n(i - 1, 2) + T^n(i + 1, 2) \\ T^n(i - 1, 3) + T^n(i + 1, 3) \\ T^n(i - 1, 4) + T^n(i + 1, 4) \\ T_{Right} \end{bmatrix}$$

The TDMA function can be called to solve for the temperature. Once the temperature is calculated, the next row will use these values for its upper row and the zeros for the bottom row to calculate the next temperature. This process is looped over. The residual is calculated using the difference between the current iteration's Matrix and the previous iteration's Matrix. We then take the L2 norm of this residual. Once the residual is less than the tolerance, the loop stops, and we obtain the value of the temperature.

#### 4.4.2 Temperature Distribution for N = 41 Grid Points

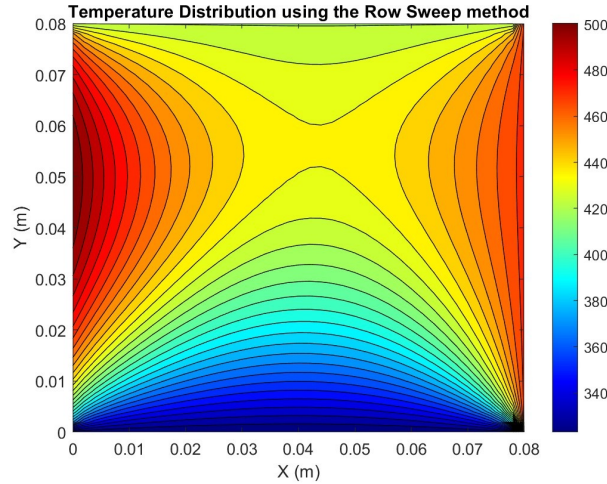


Figure 8: Temperature distribution for N = 41 grid points using Row Sweep Method



#### 4.4.3 Comparison of CPU Run Time for $N = 11, 21, 41$ grid points

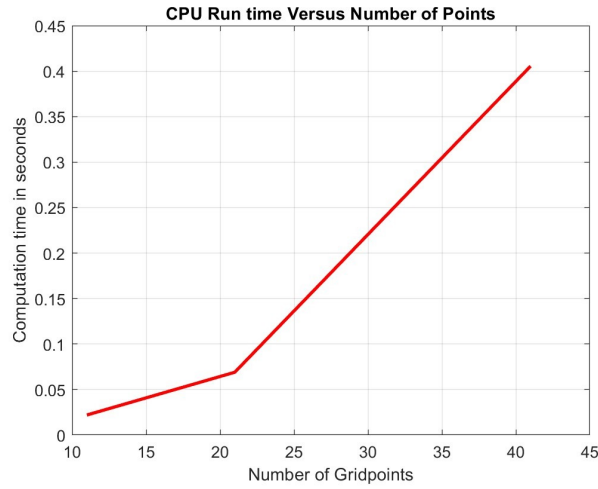


Figure 9: Variation of CPU Run Time with increase in  $N$

#### 4.4.4 Comparison of CPU Run Time for Row Sweep versus Gauss Elimination method.

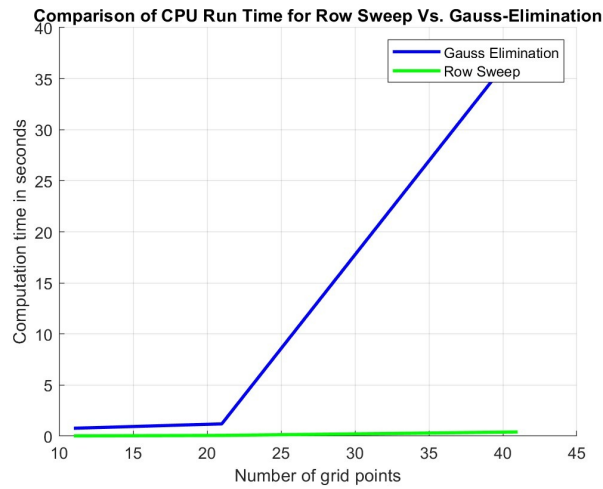


Figure 10: Comparison of CPU Run Time for different methods

Looking at the graph above, it is very clear that Gauss Elimination is an inefficient method in computing the solution, as for even smaller  $N$ , the time it takes is much more than Row Sweep method. Hence, the Row sweep method is a better method for getting the solution.

#### 4.4.5 Comparison of CPU Run Time for Row Sweep versus Gauss Seidel method.

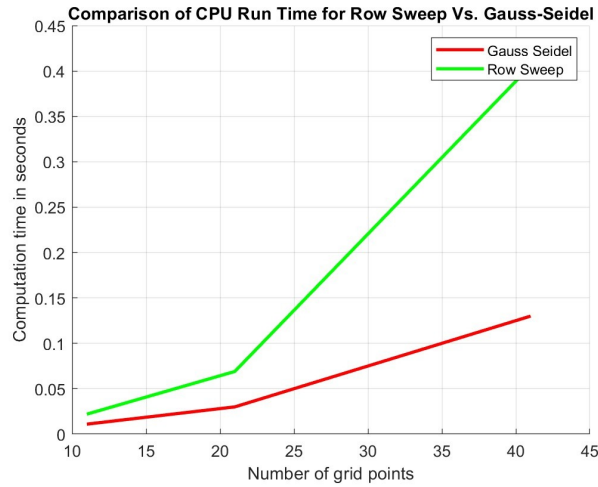


Figure 11: Comparison of CPU Run Time for different methods

Looking at the graph above, it is clear that the Gauss-Seidel method is still a better approach for obtaining the solution compared to the Row Sweep method. Although both methods are extremely fast, we observe that as  $N$  increases to 41, there is a clear increase in computation time compared to the Gauss-Seidel method. Hence, both methods are efficient, but the Row sweep method loses time in creating and computing the solution for the matrix  $A$  and Vector  $B$ , whereas the Gauss-Seidel method performs very simple calculations for the computer, making it an efficient method for obtaining the solution.

#### 4.4.6 Calculating the Residual

Calculation of the residual is a crucial step in the iterative process, as it enables us to determine whether our solution is converging or diverging.

There are multiple methods of calculating the residual. One method includes calculating the  $L2$  Norm of the matrix, and the second involves calculating the mean of the difference between the current iteration matrix and the previous iteration matrix.

The method I employed to calculate the residual involves calculating the mean of the difference between the current iteration matrix and the previous iteration matrix, as shown in the formula below.

$$\text{Residual} = \frac{1}{N^2} \sum_{i,j} |T_{i,j}^{(n+1)} - T_{i,j}^{(n)}|$$

- $n$  denotes the previous iteration,  $n + 1$  denotes the current iteration (or time step).
- $N$  denotes the number of grid points.

#### 4.4.7 Comparison of Residual versus number of iterations of Gauss-Seidel and Line by Line method

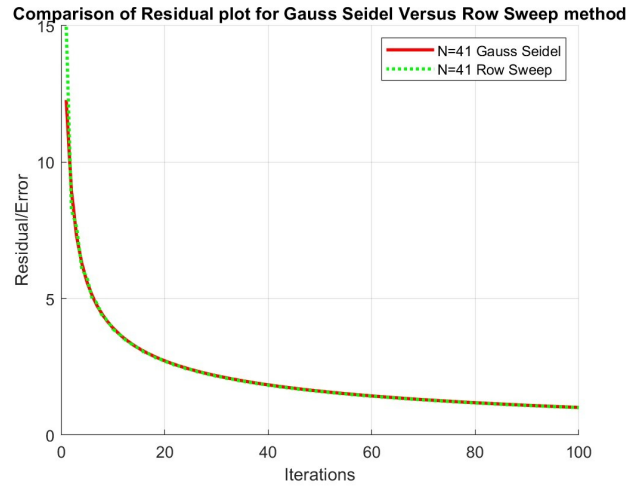


Figure 12: Residual Comparison between Gauss-Seidel and Line by Line method

- Analysing the graph above, it is clear that for  $N = 41$ , the initial residue for the row sweep method is greater than that of the Gauss-Seidel method.
- As the iterations increase, the change in the residue is greater for the row sweep method than for the Gauss-Seidel method, indicating that the convergence rate for the row sweep method is higher.
- The total number of iterations required for convergence for  $N = 41$  in the case of the Row sweep method was 2756, and in the case of Gauss Gauss-Seidel method, it was 2764, which also suggests that the row sweep method converges faster.
- Though the convergence rate of row sweep method is more than the Gauss Seidel method the row sweep method loses time in creating and computing the solution for the matrix  $A$  and Vector  $B$ , whereas the Gauss-Seidel method performs very simple calculations for the computer, making it an efficient method for obtaining the solution.

#### 4.5 Alternating Direction Implicit (ADI) Method

The Alternating Direction Implicit (ADI) Method is also a type of line-by-line method, in which we employ the row sweep method and the column sweep method alternatively. This method is efficient as it helps converge the solution faster than if we used only the row sweep method or the column sweep method.

#### 4.5.1 Comparison of Residual versus number of iterations for row sweep, column sweep and ADI method

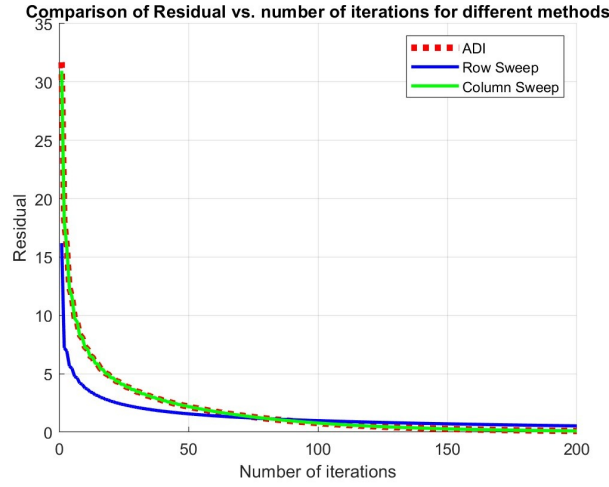


Figure 13: Residual Comparison between different Line by Line methods

- In our problem statement, the column sweep converges faster than the row sweep method, as the column sweep takes only 768 iterations, whereas the row sweep method takes 2747 iterations to converge, which is almost 3.5 times more. Hence, the convergence rate of column sweep is hence more than the row sweep.
- The ADI method employs both the row sweep and column sweep, and since the column sweep's convergence rate is higher, we can see that the graph for ADI almost coincides with the column sweep's graph.
- The ADI method requires 764 iterations, which is slightly fewer than the number of iterations that the column sweep method takes. This is the advantage of this method, as it alternates between the row and column sweep methods, thereby reducing the total number of iterations.

## 5 Concluding Remarks

After using multiple methods, the most efficient methods were the iterative methods. Although direct methods like Gauss Elimination provide an accurate solution, the accuracy of iterative solvers is still quite high, still extremely fast.

When comparing the line-by-line solvers, the ADI method converges faster than the Row sweep and Column sweep methods.

Lastly, increasing the number of grid points yields more accurate temperature values, and having a tighter tolerance level after a certain point has minimal effect on accuracy, but it increases computation time.

## 6 Acknowledgments

I would like to express my gratitude to Professor Dilip Sundaram for teaching us the concepts of Computational Fluid Dynamics in a clear and concise manner, and for assigning this project to practically test our understanding of the concepts we were taught. Along with this, I would like to thank our Teaching Assistants, Mr Pardha Sai Mutyala and Mr Soham Pathak, for teaching us different algorithms during the tutorial sessions and helping me clarify any questions that arose while completing this project.

## 7 References

1. <https://www.geeksforgeeks.org/dsa/gaussian-elimination/>
2. <https://en.wikipedia.org/wiki/Gauss>